

Advanced Parallel Programming & Architecture

Laboratory Examination & Silicon Profiling Worksheet: Empirical Microarchitectural Bottlenecks
Session Duration: 120 Minutes | Total Evaluation: 100 Points | Mode: Individual Bare-Metal Execution

Student Full Name	Altynbek Dlnmukha,ed
Student Identification Number	230103113
Date & Laboratory Session	18.09.2026
Workstation / Laptop Make & Model	Apple m1
Processor SKU / SoC Model	e.g., AMD Ryzen 7 5800H / Apple M2 Pro / Intel Core i7-12700H
Core Topology (Physical P-cores / E-cores)	Physical Cores: 8 Logical SMT Threads: 8
Cache Hierarchy Capacity	L1: 64 KB L2: 4 MB L3: - MB
Host OS & Python Runtime	Mac os

■ MANDATORY LABORATORY PROTOCOL & HARDWARE FIDELITY NOTICE

This laboratory examination evaluates empirical microarchitectural phenomena directly on your machine's bare-metal silicon. LLMs and AI coding assistants cannot predict the real runtime numbers, thermal throttling curves, or cache eviction latencies of your specific processor SKU. Any submission containing fabricated, perfectly theoretical, or non-empirical data will receive an automatic zero. You are required to submit this completed worksheet along with your raw terminal logfile 'results_<student_id>.txt' before the 120-minute timer elapses.

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Recommended Allocation: 35 Minutes | Task Weight: 25 Points | Concepts: Strong Scaling, SMT, Amdahl Fraction

Amdahl's Law defines the theoretical upper bound of speedup for a fixed-size workload when scaling across multiple processors:

$$S(p) = 1 / [(1 - P) + (P / p)]$$

where S(p) is the speedup ratio T(1)/T(p), P is the strictly parallelizable fraction of the algorithm, and p is the number of active worker cores. In physical silicon, speedup curves diverge sharply from theoretical models due to process initialization overhead (fork/spawn latency), inter-process communication (IPC) serialization, OS context switching, and resource contention on Simultaneous Multithreading (SMT/Hyper-Threading) logical cores.

■ EXECUTION PROTOCOL

Create a script named 'task1_amdahl.py' using the benchmark code below. Execute the script from your terminal: 'python3 task1_amdahl.py'. Record the single-process baseline T(1) and the measured multi-core execution times across all 6 core allocations.

```

import time
import os
import multiprocessing as mp

def cpu_intensive(chunk_size):
    acc = 0
    for i in range(chunk_size):
        acc += (i % 7) * (i % 11)
    return acc

def run_bench(workers, total_items=40_000_000):
    chunk = total_items // workers
    start = time.perf_counter()
    with mp.Pool(processes=workers) as pool:
        pool.map(cpu_intensive, [chunk] * workers)
    return time.perf_counter() - start

if __name__ == "__main__":
    cores = [1, 2, 4, 8, 12, 16]
    base = run_bench(1)
    print(f"OS Reported Logical Cores: {os.cpu_count()}")
    print(f"Single Process Baseline Time T(1): {base:.4f}s\n")
    print("Cores | Time(s) | Observed Speedup | Theoretical Linear")
    print("-" * 55)
    for c in cores:
        t = run_bench(c)
        speedup = base / t
        print(f"{c:5d} | {t:7.4f} | {speedup:16.2f}x | {c:18.2f}x")

```

Worker Cores (p)	Measured Time T(p) [s]	Observed Speedup S(p)	Ideal Linear Speedup	Parallel Efficiency [S(p)/p * 100]
p = 1 (Baseline)	1.9569	1.03x	1.00x	103.0%*
p = 2	1.0313	1.96x	2.00x	98.0%
p = 4	0.5598	3.60x	4.00x	90.0%
p = 8	0.4862	4.15x	8.00x	51.9%
p = 12	0.5186	3.89x	12.00x	32.4%
p = 16	0.5586	3.61x	16.00x	22.6%

[Q1.1] Identify the exact inflection point (core count p*) where your observed speedup curve flattens or degrades. Correlate this directly to your machine's physical hardware topology (distinguish between physical Performance cores,

Efficiency cores, and logical SMT hyperthreads). (8 Points)

Student Response & Architectural Justification:

The inflection point on my machine happens at $p^* = 8$ cores. This makes sense because my MacBook has an Apple M1 chip, which has 8 physical cores total (4 Performance cores and 4 Efficiency cores) and no SMT/Hyper-Threading like Intel CPUs. So there are no "extra" logical threads beyond the 8 physical ones.

Looking at my table, efficiency actually starts dropping earlier, between $p=4$ and $p=8$ (from 90% down to 52%), even though I still have physical cores available. This is because the first 4 workers run on the fast P-cores, but workers 5 to 8 get pushed onto the slower E-cores, which have lower clock speed and IPC. So the speedup still grows, just not linearly, because the extra cores are weaker.

After $p=8$, speedup actually gets worse (from 4.15x down to 3.89x and then 3.61x at $p=16$). This is because there are only 8 physical cores in total, so running 12 or 16 processes forces the OS to context-switch multiple processes on the same core, which adds scheduling overhead instead of doing more real work.

[Q1.2] Using your empirical measurements for $T(1)$ and $T(4)$, calculate the exact value of P (the parallel fraction) of this workload on your silicon. Show the complete algebraic derivation: $S(4) = T(1)/T(4) = 1 / [(1 - P) + P/4]$. (9 Points)

Using my measured values $T(1) = 2.0165s$ and $T(4) = 0.5598s$:

$$S(4) = T(1) / T(4) = 2.0165 / 0.5598 = 3.603$$

Now I plug this into Amdahl's Law and solve for P :

$$S(4) = 1 / [(1 - P) + P/4]$$

$$3.603 = 1 / [(1 - P) + 0.25P]$$

$$3.603 = 1 / [1 - 0.75P]$$

$$1 - 0.75P = 1 / 3.603 = 0.2775$$

$$0.75P = 1 - 0.2775 = 0.7225$$

$$P = 0.7225 / 0.75 = 0.9633$$

So $P \approx 0.963$, meaning about 96.3% of the workload is parallelizable, and only about 3.7% is strictly serial (this small serial part comes from things like process spawning overhead and collecting/merging results with `pool.map`).

[Q1.3] Compute the Asymptotic Speedup Limit $S_{\max} = \lim(p \rightarrow \infty) S(p) = 1 / (1 - P)$ using your calculated P from Q1.2. Explain what this limit implies if your department purchased a 128-core server node to run this specific

workload. (8 Points)

Using $P = 0.9633$ from Q1.2:

$$S_{\text{max}} = 1 / (1 - P) = 1 / (1 - 0.9633) = 1 / 0.0367 \approx 27.25x$$

This means that even with an infinite number of cores, this workload could never run faster than about 27.25 times the single-core time. So if my department bought a 128-core server for this specific workload, it would be a huge waste of money — we would never get anywhere close to 128x speedup, because the small serial fraction (3.7%) becomes the bottleneck no matter how many cores we add.

What's even more interesting is that my real data shows the actual limit is reached way before the theoretical one. The best speedup I measured was only 4.15x at $p=8$, and after that performance got worse, not better. This shows that in practice, the bottleneck isn't just the serial fraction predicted by Amdahl's Law — it's also limited by real physical hardware constraints (only 8 physical cores, and heterogeneous P-cores/E-cores), which cause the curve to flatten and degrade much earlier than the theoretical 27.25x ceiling.

TASK 2: CACHE COHERENCE & THE FALSE SHARING TRAP

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: MESI Protocol, 64-Byte Cache Lines, Bus Invalidation

Modern symmetric multiprocessing (SMP) architectures maintain hardware cache coherency via snooping protocols such as MESI (Modified, Exclusive, Shared, Invalid). Data moves between main memory (DRAM) and CPU caches in discrete 64-byte blocks called cache lines. False Sharing occurs when two independent threads executing on physically separate CPU cores continuously write to distinct variables that happen to reside on the same 64-byte cache line. Although no logical data race exists, each write forces the core's cache controller to broadcast an Invalidation signal across the interconnect bus, evicting the line from the other core's L1/L2 cache and causing massive pipeline stalls.

■ EXECUTION PROTOCOL

Create 'task2_falsesharing.py'. Run the benchmark 3 separate times to eliminate background OS scheduling noise. The test compares 4 threads writing to adjacent list indices vs. 4 threads writing to padded indices separated by 16 elements ($16 * 8$ bytes = 128 bytes, guaranteeing placement on distinct 64-byte cache lines). Record the median execution times.

```

import threading
import time

ITERATIONS = 5_000_000

def worker_adjacent(shared_list, index):
    for _ in range(ITERATIONS):
        shared_list[index] += 1

def worker_padded(shared_list, index):
    # Offset by 16 integers (16 * 8 bytes = 128 bytes > 64-byte cache line)
    padded_idx = index * 16
    for _ in range(ITERATIONS):
        shared_list[padded_idx] += 1

def run_test(target_fn, size):
    arr = [0] * size
    threads = [threading.Thread(target=target_fn, args=(arr, i)) for i in range(4)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return time.perf_counter() - start

if __name__ == "__main__":
    t_adjacent = run_test(worker_adjacent, size=4)
    t_padded = run_test(worker_padded, size=64)
    print(f"Adjacent Indices (False Sharing): {t_adjacent:.4f}s")
    print(f"Padded Indices (Cache-Aligned): {t_padded:.4f}s")
    print(f"Slowdown Factor: {t_adjacent / t_padded:.2f}x")

```

```

(base) dimasaltynbek@MacBook-Pro-Dimas Desktop % python3 task2_falsesharing.py
Adjacent Indices (False Sharing): 0.7385s
Padded Indices (Cache-Aligned): 0.7692s
Slowdown Factor: 0.96x
(base) dimasaltynbek@MacBook-Pro-Dimas Desktop % python3 task2_falsesharing.py >> resultsTask2_230103113.txt
(base) dimasaltynbek@MacBook-Pro-Dimas Desktop % python3 task2_falsesharing.py
Adjacent Indices (False Sharing): 0.7304s
Padded Indices (Cache-Aligned): 0.7292s
Slowdown Factor: 1.00x
(base) dimasaltynbek@MacBook-Pro-Dimas Desktop % python3 task2_falsesharing.py
Adjacent Indices (False Sharing): 0.7569s
Padded Indices (Cache-Aligned): 0.7363s
Slowdown Factor: 1.03x

```

Memory Layout Configuration	Trial 1 [s]	Trial 2 [s]	Trial 3 [s]	Median Time [s]	Slowdown Penalty Multiplier
Adjacent Indices (False Sharing Active)	0.7385	0.7304	0.7569	0.7385s	— [Baseline]
Padded Indices (Cache-Line Aligned)	0.0.7692	0.7292	0.7363	0.1.00x	

Slowdown Factor	0.96x	1.00x	1.03x	≈1.00x	
-----------------	-------	-------	-------	--------	--

[Q2.1] Calculate the exact memory footprint of a 64-bit integer pointer/reference in CPython. Mathematically prove why an index stride of 16 guarantees that adjacent threads never access memory within the same 64-byte L1 cache line. (8

Points)

In CPython, a reference (pointer) is 8 bytes on a 64-bit system, since it's a memory address and 64-bit systems use 8-byte addresses. A standard L1 cache line is 64 bytes wide.

If I use a stride of 16 elements between indices (index * 16), the memory distance between two adjacent thread's slots is:

16 elements × 8 bytes = 128 bytes

Since 128 bytes > 64 bytes (the cache line size), this guarantees that consecutive threads never write into slots that fall inside the same 64-byte cache line. Even if the very first byte of a line lines up with one thread's slot, the next thread's slot is at least 128 bytes away, so it must be on a completely different cache line (or even skip one line entirely). Mathematically: if slot *i* starts at address *A*, slot *i*+1 starts at *A* + 128, and one cache line only covers addresses [*A*, *A*+64), so *A*+128 is guaranteed to be outside that line.

[Q2.2] Trace the MESI state transitions for the shared cache line containing array elements. When Core 0 executes an increment on index 0 and Core 1 subsequently executes an increment on index 1, describe the bus invalidation messages and state shifts (Shared -> Modified -> Invalid). (9 Points)

When Core 0 writes to index 0, the cache line containing index 0 (and index 1, since they're adjacent in memory) moves into Core 0's cache in the Modified (M) state, because Core 0 is the only one that has written to it.

Now when Core 1 tries to increment index 1, which lives on the exact same 64-byte cache line, Core 1's cache controller has to fetch that line. Since Core 0 currently holds it in Modified state, the MESI protocol forces Core 0 to write the line back and downgrade its own copy, and Core 1 gets the line loaded (this can go through a state like Shared briefly, or directly to Invalid on Core 0's side once Core 1 writes to it).

Then, as soon as Core 1 writes to index 1, the line becomes Modified in Core 1's cache, and Core 0's copy is immediately marked Invalid. If Core 0 then wants to write index 0 again, the same thing happens in reverse. So the line keeps bouncing: Modified → Invalid → Shared/Modified → Invalid, back and forth between the two cores, even though the cores are logically touching completely different variables (index 0 vs index 1). Every single one of these transitions requires a bus invalidation message and a fresh cache line fetch, which is what makes false sharing so expensive.

(In my actual results, this effect barely showed up, because Python's threading module is limited by the GIL — only one thread executes Python bytecode at a time, so the two threads never truly write to memory simultaneously from two separate physical cores the way C/C++ threads would. That's likely why my Slowdown Factor came out close to 1.00x instead of showing a real penalty.)

[Q2.3] In compiled systems programming, manual array padding wastes memory. Describe two industry-standard compile-time or VM-level mechanisms used to eliminate false sharing (e.g., C++11 'alignas(64)', Java '@Contended', or Go struct field padding). (8 Points)

Two industry-standard mechanisms to avoid false sharing without manually padding arrays:

1. C++11 `alignas(64)` — this lets the compiler align a variable or struct member to a 64-byte boundary automatically, forcing it to start at the beginning of its own cache line, so it never shares a line with another variable. The compiler handles the padding calculation instead of the programmer doing it by hand.
2. Java's `@Contended` annotation — when applied to a field, the JVM automatically inserts padding around that field so it lands on its own cache line, specifically to prevent false sharing between threads that access different fields of the same object concurrently.

(Go also solves this at compile time, by letting the programmer add explicit padding fields in a struct definition so hot fields used by different goroutines land on separate cache lines — though this is closer to manual padding than the automatic mechanisms above.)

TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: Lock Contention, Race Conditions, Map-Reduce Partitioning

Mutual exclusion primitives (locks/mutexes) prevent data corruption by serializing critical sections. However, coarse grained locks introduce catastrophic performance penalties: user-space thread suspension, OS kernel context switches (futex system calls), and cache line invalidation across cores competing for the lock variable. In distributed and concurrent systems architecture, the foundational principle is: 'The fastest lock is no lock.'

■ EXECUTION PROTOCOL

Create 'task3_sync.py'. Run the benchmark comparing the UnsafeCounter (unprotected shared variable leading to silent data corruption) against the LockedCounter (mutex-protected serialization). Record the corrupted value and the elapsed runtimes.

```
import threading
import time
```

```
TOTAL_OPS = 2_000_000
NUM_THREADS = 4
```

```

class UnsafeCounter:
    def __init__(self): self.val = 0
    def inc(self): self.val += 1

class LockedCounter:
    def __init__(self):
        self.val = 0
        self.lock = threading.Lock()
    def inc(self):
        with self.lock:
            self.val += 1

def bench(counter_type):
    c = counter_type()
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    def work():
        for _ in range(ops_per_thread):
            c.inc()
    threads = [threading.Thread(target=work) for _ in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return c.val, time.perf_counter() - start

if __name__ == "__main__":
    val_unsafe, t_unsafe = bench(UnsafeCounter)
    val_locked, t_locked = bench(LockedCounter)
    print(f"Unsafe: Value = {val_unsafe:,} / {TOTAL_OPS:,} | Time: {t_unsafe:.4f}s")
    print(f"Locked: Value = {val_locked:,} / {TOTAL_OPS:,} | Time: {t_locked:.4f}s")
    print(f"Contention Cost Multiplier: {t_locked / t_unsafe:.2f}x")

```

Counter Architecture	Target Sum	Actual Empirical Sum	Data Corruption Defect Count	Execution Time [s]	Contention Overhead
Unsafe Accumulator (Race Condition)	2,000,000	2,000,000	0	0.0962	1.00x [Baseline]
Locked Accumulator (threading.Lock)	2,000,000	2,000,000	0 (Protected)	0.2659	2.76x
Lockless (Thread-Local)	2,000,000	2,000,000	0	0.0517	-

[Q3.1] Analyze your Unsafe Accumulator output. What was your exact corrupted count? Explain the low-level read modify-write machine code sequence (LOAD, ADD, STORE) that caused lost updates between overlapping thread executions. (7 Points)

So my corrupted count actually turned out to be 2,000,000/2,000,000, no corruption at all in my case. I was expecting to see some lost updates though, because `self.val += 1` isn't really one atomic operation on the machine code level. It's actually 3 steps: LOAD (grab the current value from memory), ADD (add 1 to it), STORE (write it back). If two threads are doing this at the same time and their steps overlap, one of the updates can just get lost. Like if both threads read `val=100` at basically the same moment, before either one writes back, they'll both compute 101 and both write 101 — so even though 2 increments happened, `val` only went up by 1.

The reason I didn't see this happen is probably the GIL. CPython only lets one thread run Python bytecode at a time, and it doesn't context switch after every single instruction, so most of the time the LOAD-ADD-STORE sequence just finishes without getting interrupted. It's not guaranteed by the language though, it's just how CPython happens to behave — this same code could break on a different interpreter or with more threads/contention.

■ ARCHITECTURAL REDESIGN CHALLENGE: LOCKLESS PARTITIONING

Redesign the accumulation algorithm without using any Mutex locks. Requirements:

1. Correctness: The final result must deterministically equal exactly 2,000,000 every single run.
2. Performance: Total execution time must be strictly FASTER than `LockedCounter` by at least 2.0x. Hint: Use thread-local accumulation buffers (Map-Reduce pattern) where each thread writes exclusively to its own private state, followed by a single reduction stage after threads join.

[Q3.2] Provide your complete Python implementation of the Lockless Thread-Local Accumulator below. Record your measured execution time and calculate the speedup factor achieved over `LockedCounter`. (11 Points)

```
import threading
import time

TOTAL_OPS = 2_000_000
NUM_THREADS = 4

def bench_lockless():
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    local_results = [0] * NUM_THREADS

    def work(thread_id):
        local_count = 0
        for _ in range(ops_per_thread):
            local_count += 1
        local_results[thread_id] = local_count

    threads = [threading.Thread(target=work, args=(i,)) for i in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    elapsed = time.perf_counter() - start

    total = sum(local_results)
    return total, elapsed
```

```
if __name__ == "__main__":  
    val_lockless, t_lockless = bench_lockless()  
    print(f"Lockless: Value = {val_lockless:,} / {TOTAL_OPS:,} | Time: {t_lockless:.4f}s")
```

I got 0.0517s for this vs 0.2659s for LockedCounter, so:

Speedup = 0.2659 / 0.0517 ≈ 5.14x

which is way more than the 2.0x we needed. And the value comes out correct every time (2,000,000/2,000,000) since no thread ever touches another thread's data while running.

[Q3.3] Why is eliminating shared mutable state (via thread-local buffers and reduction) fundamentally superior to optimizing lock primitives (e.g., adaptive spinlocks or read-write locks) in high-throughput concurrent systems? (7 Points)

Basically, getting rid of shared state fixes the actual problem instead of just making it a bit less painful. Even a really optimized lock still has to force threads to wait for each other at some point — that's the whole point of a lock. So no matter how fast you make it, there's still serialization happening, and possibly context switches if the OS has to put a thread to sleep while it waits.

With thread-local buffers there's nothing to wait for. Each thread just writes to its own private slot, so there's no cache line being fought over between cores and no coordination needed at all while the threads are running. The only time anything gets combined is right at the end, once, during the reduction — and that's basically free compared to doing it 500,000 times per thread like the lock does.

My numbers back this up too — LockedCounter took 0.2659s because it's paying the lock cost on every single increment, but the lockless version only took 0.0517s because it only pays that cost once, at the end.

TASK 4: THE MEMORY WALL VS. COMPUTE SATURATION

Recommended Allocation: 25 Minutes | Task Weight: 25 Points | Concepts: Arithmetic Intensity, Roofline Model, DRAM Bus Saturation

The Roofline Model characterizes computational performance by relating Arithmetic Intensity (FLOPs executed per byte of DRAM transferred) to the physical limitations of the host architecture. When a workload has high arithmetic intensity, execution speed is bound by the CPU's ALU/FPU pipelines (Compute-Bound). When arithmetic intensity is low, multiple cores saturate the off-chip memory bus and memory controllers (Memory-Bound). In memory-bound regimes, adding more parallel workers does not increase throughput; it merely increases memory bus contention and queueing latency.

■ EXECUTION PROTOCOL

Create 'task4_roofline.py'. Execute both benchmark suites across 1, 2, and 4 worker processes. The compute-bound suite performs 25,000,000 floating-point operations in registers with zero memory transfers. The memory-bound suite streams 50,000,000 64-bit float arrays (~400 MB per worker) from main memory with minimal math.

```

import time
import multiprocessing as mp
import numpy as np

def compute_heavy(n):
    x = 1.0001
    for _ in range(n):
        x = (x * 1.000001) + 0.00001
    return x

def memory_heavy(size):
    arr = np.ones(size, dtype=np.float64)
    arr = arr * 2.0 + 1.0
    return arr[0]

def run_suite():
    print("=== Compute-Bound Suite (Register Math) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(compute_heavy, [25_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

    print("\n=== Memory-Bound Suite (DRAM Bandwidth Saturation) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(memory_heavy, [50_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

if __name__ == "__main__":
    run_suite()

```

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]
w = 1 Worker (Baseline)	0.7886	1.00x	0.6344	1.00x
w = 2 Workers		1.11x	0.7229	1.14x

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]

w = 4 Workers	1.1437	1.45x	1.2985	2.05x
---------------	--------	-------	--------	-------

[Q4.1] Analyze the scaling divergence between the two workloads. Why does running 4 parallel compute tasks complete in roughly the same elapsed time as 1 task, whereas running 4 parallel memory tasks causes total execution time to spike dramatically? (9 Points)

So the compute part barely got slower going from 1 to 4 workers (0.79s → 1.14s, about 1.45x), while the memory part got a lot worse (0.63s → 1.30s, about 2.05x). Makes sense once you think about where the bottleneck actually is in each case.

For the compute-bound task, everything's happening inside the CPU registers, nothing has to go out to RAM, so each worker/core can just crunch numbers on its own without competing for anything shared. 4 processes running at once should be close to 4 separate cores just doing math independently, so it should scale pretty well.

For the memory-bound task, every worker is pulling big numpy arrays out of DRAM (~400MB each), and all cores share the same memory bus and memory controller. When you only have 1 worker, it gets the whole bus to itself. But when you run 4 workers at once, they're all trying to pull hundreds of MB through the same physical bus simultaneously, so they end up waiting on each other — the bus becomes the bottleneck instead of the CPU. That's why adding more workers doesn't help and actually makes total time worse, since now you've got queuing/contention on top of the same amount of data movement.

(My compute numbers scaled a bit worse than I expected too — I think this is because the process spawn overhead on macOS starts to matter more when the actual work per worker is relatively small, so some of that "scaling" is really just multiprocessing.Pool startup cost, not pure compute contention.)

[Q4.2] Investigate your host machine's physical memory specifications (e.g., DDR4-3200 vs DDR5-4800, Single-Channel vs Dual-Channel, or Apple Unified LPDDR5). Calculate or look up your laptop's theoretical peak memory bandwidth (in GB/s). Explain how memory bus width limits multi-core scalability. (8 Points)

My M1 has LPDDR4X memory running at 4266 MT/s, on a 128-bit (16-byte) wide memory bus split across 2 channels (64-bit per channel). This is Apple's Unified Memory architecture, so this same memory pool and bandwidth is shared between the CPU, GPU, and Neural Engine — not just the CPU cores.

The theoretical peak bandwidth works out to about 68.25 GB/s (this is a published Apple spec, not something I calculated by hand).

This directly explains why my memory-bound benchmark got worse with more workers: 68.25 GB/s has to be split between however many workers are actively streaming data at once. With 1 worker, it can use close to the full bandwidth. With 4 workers pulling ~400MB arrays simultaneously, they're all fighting over that same fixed 68.25 GB/s pipe, so each one effectively gets less bandwidth than before, and total time goes up instead of down. Basically the memory bus width and total bandwidth put a hard ceiling on how much data-movement work can happen in parallel, no matter how many CPU cores you throw at it — this is different from compute-bound work, which isn't limited by the bus at all since it doesn't need to move data in and out of RAM.

[Q4.3] Suppose you are deploying a machine learning embedding pipeline that transforms high-dimensional feature vectors. Based on your Task 4 findings, would optimizing your code with CUDA GPU kernels or CPU OpenMP pragmas yield greater speedup if your workload is memory-bandwidth saturated? Justify your architectural choice. (8 Points)

If the ML embedding pipeline is memory-bandwidth bound (which is what my Task 4 results suggest for this kind of workload — lots of data movement, not much actual math per byte), then I'd say CUDA GPU kernels would give a bigger speedup than CPU OpenMP.

The reasoning is basically: OpenMP just adds more CPU threads/cores, but like I saw in my memory-bound test, more workers doesn't help once you're bottlenecked on the memory bus — you're still stuck sharing the same ~68 GB/s pipe between more competitors, so OpenMP would probably run into the exact same wall I hit in Task 4.

GPUs, on the other hand, are built specifically for high memory bandwidth — a modern GPU can have several hundred GB/s to even TB/s of bandwidth, way more than a CPU's memory bus, specifically because GPU workloads (like ML) are usually memory-bandwidth-hungry to begin with. So moving the embedding pipeline to a GPU wouldn't just add more parallel compute units, it would also remove the memory bottleneck that's actually causing the slowdown in the first place. Since my data showed the memory-bound case degrading way more than the compute-bound case, I think the smart move here is to fix the actual bottleneck (bandwidth) rather than adding more CPU cores that will just end up waiting on the same bus like they did in my test.

EVALUATION RUBRIC & INSTRUCTOR VERIFICATION SIGN-OFF

Total Laboratory Marks: 100 Points | Submission Verification & Academic Audit

Task 1: Amdahl Scaling	Complete empirical core sweep table, valid inflection identification, rigorous algebraic derivation of P, and asymptotic limit analysis.	25	
Task 2: Cache Coherence	Three-run trial data for adjacent vs padded layouts, mathematical verification of 64-byte cache boundary, and MESI protocol trace.	25	
Task 3: Sync & Lockless	Accurate race condition defect capture, lock contention measurement, functioning lockless map-reduce redesign with >2x verified speedup.	25	
Task 4: Roofline Analysis	Compute vs Memory scaling data table, identification of DRAM bus saturation, host memory channel analysis, and architectural justification.	25	
TOTAL EVALUATION	Comprehensive Silicon Profiling & Architectural Systems Mastery	100	

Instructor / TA Walk-Around Verification Check:	Laboratory Submission Audit Timestamp:
☐ Task 3 Unsafe Corruption Verified ☐ Task 2 Cache Slowdown Verified ☐ Terminal Logfile 'results_<id>.txt' Uploaded	Examiner Signature: _____ Date:

APPENDIX: HARDWARE TOPOLOGY QUERY COMMANDS

Use these native OS terminal commands to extract your exact microarchitectural parameters

Linux (Ubuntu/Debian)	lscpu grep -E 'model name cpu cores' /proc/cpuinfo	lscpu -C sudo dmidecode --type memory grep Speed
macOS (Apple Silicon/Intel)	sysctl -n machdep.cpu.brand_string sysctl hw.perflevel0.physicalcpu	sysctl hw.l1dcachesize hw.l2cachesize system_profiler SPMemoryDataType
Windows (PowerShell / CMD)	Get-CimInstance Win32_Processor Select Name, NumberOfCores wmic cpu get name,NumberOfCores,NumberOfLogicalProcessors	Get-CimInstance Win32_CacheMemory wmic memorychip get speed,capacity,devicelocator

Bonus Task

Copy-paste this code. Do not change the logic. Run it across thread/process counts from 1 to 32, graph your actual execution times, and answer the autopsy questions.

import time

from multiprocessing import Pool

import threading

def cpu_work(n):

Pure CPU crunch

count = 0

for i in range(n):

count += i * i

return count

Run both experiments with total_work = 50_000_000 split across T

workers # Test T in [1, 2, 4, 8, 16, 32]

Deliverables:

- 1. The Empirical Curve:** Plot T (workers) on the X-axis vs. Execution Time on the Y axis for both threading.Thread and multiprocessing.Pool.
- 2. The Inflection Cliff:** Identify the exact number of workers where adding more workers makes the program *slower*.
- 3. The Architectural Autopsy:**

- Why does Python's threading curve remain virtually flat or get worse, while multiprocessing scales? (Forces them to articulate the GIL).
- Look up your laptop's CPU model (e.g., Apple M-series, Intel Core i5/i7, AMD Ryzen). How many *physical performance cores* do you have vs. *logical threads*? ◦ Pinpoint the exact point on your graph where physical core saturation occurs, and explain the performance drop when $T > \text{physical cores}$ using context-switching overhead.